

An Investigation Into Methods for the Detection of Malicious Software Using Techniques from Machine Learning

Henry Williams¹

¹School of Engineering and Informatics, The University of Sussex

Abstract

In this report, we outline our plan for our investigation into approaches for the detection of malware which utilise techniques from machine learning. Our investigation will rigorously evaluate these approaches on a custom dataset of windows executables. In addition to this, we will also investigate any potential means by which we can improve these approaches, and how effectively the models we investigate handle obfuscation.

1 Introduction

Malicious software, otherwise known as malware, are a form of computer software which intentionally perform unwanted or harmful actions on a system, without the users consent [19]. Security researchers consider these programs to be one of the most widespread risks faced by both individuals and organisations. In order to mitigate these risks, a number of systems to detect and flag malware before it can infect a system have been proposed [7, 3, 27]. However, these rely on a database of known malicious software, rendering them ineffective against new malware. Furthermore, the developers of these programs will typically employ techniques to confuse these scanners, known as obfuscation [40], further reducing their efficacy.

Machine learning enables for more accurate, and robust detection of harmful programs, by learning characteristics indicative of a programs' ill intent. This greatly reduces the risk posed by such programs [16, 22, 17, 37], and has been an active area of computer security research.

Our investigation will provide a rigorous evaluation of various approaches utilising machine learning to discriminate between both malicious, and non-malicious software, which we refer to as benign software throughout this report. Our experiments will take place on a custom dataset containing benign, and malicious executables in the Windows Portable Executable file format, and we will test both static and dynamic approaches.

2 Background

2.1 Malware

Malware are software which perform unwanted or harmful actions on a host system. Security researchers often classify these programs into various categories based on their behaviour which are explored in Section 2.3.

As we become increasingly reliant on digital infrastructure in our daily lives, mitigating the risk posed by malware is of the utmost importance. For example, in 2017, ransomware known as WannaCry begun spreading throughout the internet, infecting an estimated 300,000 systems [1]. One of the most prominent victims of this attack was the National Health Service, who were unable to access patient information, and make use of vital medical equipment for its duration [26]. Furthermore, it is estimated that the damages to systems infected by WannaCry costed the global economy \$4 Billion [1]. This shows not only the potential financial impact of these programs, but the potential human cost as well, and why preventing such an attack is so important.

2.2 Windows Portable Executable File Format

Executables targeting the Windows operating system are written in the Portable Executable (PE) file format. These files are often not written by developers themselves as they require expert knowledge of low level programming, and instead are generated by a compiler. A high level overview of this file format is shown in Figure 1, and it has been used since Windows 95, making it incredibly widespread throughout the internet.

The file begins with a DOS (Disk Operating System) header for compatibility reasons [18], followed by the PE header, containing information necessary for its execution. This includes the number of sections and symbols, and the instruction set used by the program. The section headers contain further metadata about the sections which comprise the program, and reference their location in memory. It also stores a pointer to a table referencing any external functions used by the program, such as the Windows API, and other dynamically linked objects, known as DLLs [10].

2.3 Types of Malware

One of the earliest forms of malware, known as a *virus* is a self replicating program which modifies other executables to infect new targets. In order to spread, they rely on the user unknowingly sharing an infected file to a new host.

Unlike a virus, *Worms* propagate throughout a network without the need for user interaction, and will often make use of software exploits to infect new targets. They are considered to be extremely dangerous, as they can rapidly spread throughout a network.

Trojan Horses present themselves as legitimate software in an attempt to trick users into infecting their devices. This is one of the most widespread forms of malware in the current era, as it is far easier to trick someone into downloading software than it is to develop sophisticated software exploits.

Spyware refers to a category of malware which exfiltrate private information, such as password, cookies and internet traffic from the host machine, likely to be resold or used to further compromise the victim.

Root Kits are installed into the operating system, unlike other malware which are installed to user space. This provides the program with the highest possible level of privilege, meaning it has direct access to operating system functionality, hardware, and external peripherals. As a result of this, this forms of malware is extremely powerful, and can be extremely difficult to detect once infected.

Ransomware refer to malware which hold a system captive, typically by encrypting the files stored by the host, until a ransom is paid.

Bots are a form of malware which enable an attacker remote control of a host system. These are typically part of a larger network of infected systems, known as a *botnet*, which are often used as part of distributed denial-of-service attacks.

It is worth noting that these categories are not mutually exclusive, and malware often exists in more than one class at once.

2.4 Detection

Early attempts at malware detection rely upon the file signatures of programs given by a cryptographic hash function. This signature of an unknown program is compared against a database containing the signatures of malware, and if the hash is present within the database, then we know the program is malicious, and can be flagged as such.

However, if the program is not present within the database, it will always be treated as benign. This means that if a malware is yet to be seen within the database, it will be allowed to execute, infecting the hosts. Malware authors exploit this by making small changes to their

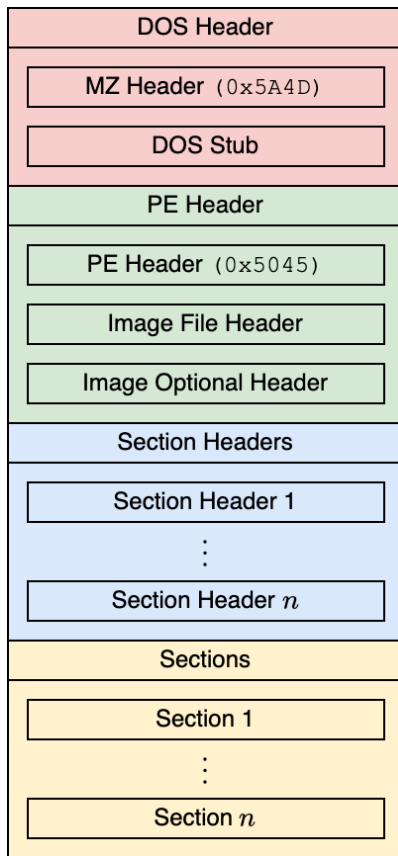


Figure 1: A high level overview of the windows portable executable file format.

software, leading to an entirely new file signature, thus tricking the detection software.

More sophisticated systems fall into one of two categories, *static*, and *dynamic*. Static detection services determine the intent of a program, without executing it, typically making them faster, at the cost of being less accurate, due to their heavily obfuscated nature. For example, they may consider metadata found within a program, which includes external subroutines, and static strings found within the program.

Dynamic Detection observes the execution of a program. This is often more powerful than static approaches, as it considers a programs' behaviour rather than its structure. However, in order to run these programs safely, an isolated environment such as a virtual machine is needed. Some forms of malware also make use of anti-analysis features, which checks the execution environment, and determines if it is inside a virtual machine before executing its malicious portion to trick dynamic analysis tools.

2.4.1 Machine Learning

Machine Learning for malware detection has been an active area of research [23, 34], as it enables software to learn common features of malware which other software may not be able to detect. This enables more accurate and resilient classifiers, and in this section, we outline a handful of approaches seen in our research.

An executable program is merely a series of instructions and data, which are interpreted by the host system. These instructions are expressed in a language known as machine code which largely follows the pattern of an instruction, known as an opcode referring to the byte used to denote the operation, followed by some operands. The approach outlined in [4] describes a classifier which makes use of n-grams, graph learning, and deep eigenspace learning to train their model, achieving 99% accuracy. In this opcode n-gram graph, edges between vertices in this graph are given by the probability of the proceeding instruction being another opcode. The authors then use a Convolutional Neural Network to learn the classification based on the first and second eigenvalues of the adjacency matrices previously generated.

However, their data set was extremely imbalanced, with approximately 1000 benign samples, and only 100 malicious samples. Furthermore, the investigation performed in [30] suggests these features are not sufficient, due to the lack of meaningful information.

Other textual features can also be extracted from executables, such as the strings stored used by the program. These can also provide a lot of information about the nature of the program which can be used to learn classification, as was performed in [35], achieving 97% detection accuracy. This approach utilises the WEKA classification interface [12].

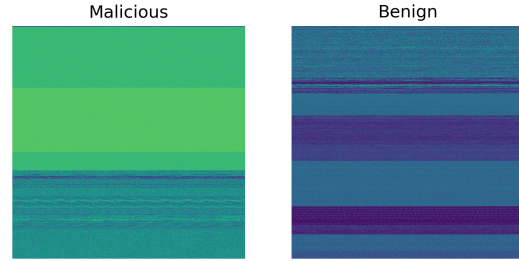


Figure 2: Malware and benign programs represented as images

The header of an executable also provides useful information, which is used by the authors of [32], to achieve an average detection accuracy of 90%. Their approach makes use of textual features found in the headers of the executable, which are preprocessed and used to train linear classification algorithms, such as Naive Bayes.

Another approach for static malware analysis is to represent the programs as images (Figure 2) either for further augmentation, or trained on directly using a Convolutional Neural Network (CNN). Approaches using this technique seem to vary in terms of efficacy, for example, in [25], the authors were able to achieve 98% classification accuracy, whereas the authors of [33] were only able to achieve 94%. This is likely due to the authors requirements of building a lightweight model for detection.

Dynamic analysis is often significantly more powerful than static analysis. For example, the authors of [22] make use of graph learning based on API calls, and their shared arguments, to develop a highly accurate detector which is resilient to most forms of obfuscation. Other approaches make use of API call sequences [14], file system modifications [28], and network communications [31].

An approach we have not yet seen in our research is Attention based models [36], which learn the relative importance of components in a sequence. These are widely used in sequence modelling, and natural language processing, and are extremely powerful. As part of our investigation, we plan on developing our own attention based model for malware detection, and comparing it to other approaches.

2.5 Obfuscation

Obfuscation, shown in Figure 3, refers to a set of techniques which make understanding a piece of software more challenging, while still performing the same operations.

2.5.1 Encrypted Malware

This form of obfuscation employs encryption algorithms, to hide its purpose.

Malware that employ this form of obfuscation are typically composed to two parts, the encrypted main body

```
def encrypt(path, crypt):
    with open(path, "rb") as f:
        d = f.read()

    d = crypt.encrypt(d)

    with open(path, "wb") as f:
        f.write(d)

def lock_dir(directory: str, crypt):
    files = os.listdir(directory)

    for file in files:
        file = os.path.join(directory, file)
        if os.path.isfile(file):
            print(f"Encrypting {file}")
            encrypt(file, crypt)
        else:
            lock_dir(file, crypt)

if __name__ == "__main__":
    crypt = CrypterKey("443-k3y-4-t3st1ng")
    lock_dir("./", crypt)
    print("Your files have been locked! Please pay 1XMR to regain access")
```

```
[.....]=[lambda getattr:([[[lambda  
    [.....].setattr('builtins',[.....],[.....])for [.....],[.....]  
in getattr(item,[.....]).chr(114)+chr(88)]]:[w?='?-Z-  
26-9;'.chr(117)+chr(117)+chr(107).import_;getattr,bytes,exec  
  
.import_('sys').setrecursionlimit(100000000)][.....][.....][.....]  
[.....].fromhex('7ad6c69d2').decode()),  
[.....].fromhex('6455e36f679257373').decode())  
([.....].fromhex('789cd5dd'-remaining payload removed for clarity-  
1133feff)).replace('\n',' ').decode()
```

Figure 3: An example of how a piece of malware may be obfuscated.

```
0x00403f35    nop
0x00403f36    jne     0x403fb3
0x00403f38    inc     eax
0x00403f39    mov     cl, 0x22 ; '"" ; 34
0x00403f3b    cmp     byte [eax], 0x2f
0x00403f3e    jne     0x403f93
0x00403f40    inc     eax
0x00403f41    cmp     byte [eax], 0x53
0x00403f44    jne     0x403f54
0x00403f46    mov     dl, byte [eax + 1]
0x00403f49    or      edx, 0x20 ; 32
0x00403f4c    cmp     dl, 0x20 ; 32
0x00403f4f    jne     0x403f54
0x00403f51    or      ebx, 2
0x00403f54    nop
```

Figure 4: Dead code insertion within a piece of malware, written as “nop”

and decryptor [39]. Upon running the program, the decryptor recovers the main body, which is then executed. This bypasses signature based detection, as each new infection uses a different key. However, the decryptor portion remains constant across instances.

Oligomorphic malware extends upon this idea, by modifying the decryptor portion of the program to further confuse detection software. This enables to evade detection far more effectively than encryption alone [40].

2.5.2 Polymorphic Code

Polymorphic code refers to a program which modifies its own instructions, but performs the same operations. Malware utilising this form of obfuscation mutate their structure upon each new infection. The mutations made to the program could include randomly re-assigning registers, inserting dead code and data, among other more advanced techniques. This form of obfuscation is incredibly powerful, as existing detection engines would struggle to find patterns between instances of polymorphic code, making detection extremely difficult.

2.5.3 Dead Code Insertion

One of the less sophisticated forms of obfuscation is to simply insert instructions which have no purpose in the

program. This could include no-operations (Figure 4), which tell the CPU to do nothing, or performing useless operations, such as modifying an unused register.

3 Aims

The aims of this project are as follows:

Firstly, we aim to curate a large, comprehensive dataset containing a diverse collection of programs. The programs found in this dataset, both malicious and benign, should exhibit a diverse range of characteristics relating to their functionality. For example, if a majority of our malicious samples are console applications, and our benign samples consist of programs with a graphical user interface, then the models we test will simply learn to classify based on the presence of a user interface.

Next, we aim to perform a rigorous evaluation of machine learning techniques for detection, trained on our dataset. We will investigate the overall accuracy of these models, in addition to their robustness to obfuscation techniques. This will require the development of an experimental framework to ensure consistency between the approaches we test.

Finally, we aim to investigate any potential avenues for new ideas and approaches that could increase the efficacy of the models we test in our experiments. This includes attention based models, as we are yet to see much research into this area.

Further work in this field could investigate how well the models we test can be applied to other domains, such as on different operating systems such as Unix based hosts.

4 Project Plan

In figure 5, we outline the expected timeline for our investigation. We begin by firstly determining which approaches we will test as part of our investigation. In parallel to this, we will also be curating the dataset we use in our experiments. This has been allocated a significant portion of time in order to ensure the dataset is

Tasks	January	February	March	April	May	June	July	August
Research								
Dataset Curation								
Static Analysis								
Text Based								
Image Based								
Dynamic Analysis								
Sandbox Development								
API Call Sequence								
Behavior								
Improvements								
Writeup								

Figure 5: Gantt Chart outlining the expected timeline of our investigation.

representative of software used by in everyday operation, and the forms of malware that may be encountered.

Upon deciding the approaches we plan to investigate, and the completion of our dataset, we will investigate static approaches to malware detection. We have separated this into two sub tasks, however, this may change over the course of our research into other potential approaches. We plan on completing our experiments in this area by mid march in order to have enough time to begin development of the sandboxing framework necessary to perform dynamic analysis.

We plan on using the Panda-RE framework [11] for our sandbox, as it enables researchers to hook into certain operating system functionality, and record the executions of programs on the host. It can also allow us to modify certain operating system functionality, such as internal APIs in order to defeat anti-analysis systems often found in malware. In our limited experimentation so far, we have been unable to get this service working, so we have allocated additional time for this task. If we are unable to get it working in a reasonable time frame however, other platforms such as CAPE [8], and Noriben [5] offer similar functionality.

Once a framework for performing dynamic analysis is achieved, we will begin our investigation into dynamic detection methods. We expect this to take longer than the static analysis investigation, and as such, we have allocated additional time.

Should we find any potential avenues for improving upon the approaches we investigate as part of our research, we will spend 2 weeks developing and evaluating it. One such avenue we plan on investigating are attention based mechanisms, as we are yet to see much research into these models for this purpose.

Finally, we have allocated at least a month to complete the write up of our investigation.

4.1 Ethical Considerations

In this investigation, we will be using a dataset containing malicious software. Because of this, there are several ethical considerations that we must consider to ensure

that our work maintains scientific and academic integrity.

The most significant consideration relates to ensuring personal and university infrastructure are not effected by the potentially dangerous software throughout our investigation. We will ensure that the execution of software within our dataset will only occur within a safe, isolated environment. To mitigate the risk of accidentally running a malicious program, we have marked all samples as non-executable, meaning that in order to run them, the user must explicitly mark them as executable before they can be executed.

It is worth noting however, that it is possible to escape these isolated environments [6]. This form of exploit is known as a sandbox escape. However, due to the complexity of such exploits, they are extremely rare, and often only available to Advanced Persistent Threats, such as nation state hackers.

Furthermore, the benign samples found within our dataset will need to be representative of software used in day-to-day operations. These programs are often copyrighted, so we must ensure they are not distributed with unauthorised users to prevent any legal issues. We have stored our dataset on a external drive, which will not be shared with anyone outside of the project.

5 Data

Existing datasets for this problem, such as the Elastic Malware Benchmark for Empowering Researchers (EM-BER) [2], consist of features extracted from executables, limiting the applications for which it can be used. Furthermore, the executables from which these features are derived are unavailable due to licensing issues [15]. As our investigation will test a variety of different approaches, which all make use of different features, we require the executables themselves to extract the required information. Other datasets we have looked at have similar issues regarding licensing, or are comprised of executables for other operating [13]. It is for these reasons that we plan on building our own custom dataset.

Our dataset will consist of executables in the windows PE file format. Malicious samples are collected from the

virussshare database [38] which contains over 90 million samples seen in the wild. It also provides detection information from a number of sources including its classification. We have downloaded over 5000 malware samples from this database, and we can always download more if it is deemed necessary.

Finding a similarly sized collection of benign software could raise some issues however. Other research in this field makes use of websites offering free, portable software such as [29] and [20]. We have also investigated package managers such as chocolatey [9] and winget [24], which provide large repositories of verified benign software that can be used freely. We have also considered the possibility of synthetic data, and scraping platforms such as GitHub should the approaches we outline above prove insufficient. In this case, synthetic data could be generated using large language models.

As malware often hides itself within a legitimate program, this could raise issues when it comes to reliable detection. To remedy this, we plan on performing pairwise similarity analysis, using the Locality Sensitive Hashing algorithm [21], to locate and remove benign samples where a similar malicious variant exists.

We will also investigate the use of augmentation techniques to modify our data. This will include the obfuscation of benign executables to make the problem more challenging. This could provide insights into how well models deal with obfuscation. Furthermore, we could categorise our dataset based the classes we discuss in Section 2.3 in order to see which models are best on certain forms of malicious software, and if any types of malware are generally easier to detect overall.

6 Conclusion

In this report, we have outlined our plan for an investigation into malware detection using techniques from deep learning. We have outlined why detection of malware is important, the different forms of malware seen in the wild, the techniques employed by malware authors to make detection more difficult, and existing attempts at malware detection. We introduce our plan for the project, and any potential ethical issues we must consider as part of our investigation. Finally, we discuss the data we plan on using as part of our investigation.

References

- [1] Maxat Akbanov, Vassilios G Vassilakis, and Michael D Logothetis. “WannaCry ransomware: Analysis of infection, persistence, recovery prevention and propagation mechanisms”. In: *Journal of Telecommunications and Information Technology* 1 (2019), pp. 113–124.
- [2] H. S. Anderson and P. Roth. “EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models”. In: *ArXiv e-prints* (Apr. 2018). arXiv: 1804.04637 [cs.CR].
- [3] Avast | Download Free Antivirus & VPN | 100% Free & Easy. URL: <https://www.avast.com/en-gb/index> (visited on 01/16/2025).
- [4] Amin Azmoodeh, Ali Dehghantanha, and Kim-Kwang Raymond Choo. “Robust Malware Detection for Internet of (Battlefield) Things Devices Using Deep Eigenspace Learning”. In: *IEEE Transactions on Sustainable Computing* 4.1 (2019), pp. 88–95. DOI: 10.1109/TSUSC.2018.2809665.
- [5] Brian Baskin. *Rurik/Noriben*. original-date: 2013-04-10T20:37:37Z. Jan. 11, 2025. URL: <https://github.com/Rurik/Noriben> (visited on 01/17/2025).
- [6] Ian Beer and Samuel Groß. *Project Zero: FORCE-DENTRY: Sandbox Escape*. Project Zero. Mar. 31, 2022. URL: <https://googleprojectzero.blogspot.com/2022/03/forcedentry-sandbox-escape.html> (visited on 01/16/2025).
- [7] *Bitdefender - Global Leader in Cybersecurity Software*. Bitdefender. URL: <https://www.bitdefender.com/en-gb/> (visited on 01/16/2025).
- [8] *CAPE Sandbox Book — CAPE Sandbox v2.2 Book*. URL: <https://capev2.readthedocs.io/en/latest/index.html> (visited on 01/17/2025).
- [9] *Chocolatey - The package manager for Windows*. Chocolatey Software. URL: <https://chocolatey.org/> (visited on 01/17/2025).
- [10] Deland-Han. *Dynamic link library (DLL) - Windows Client*. Feb. 19, 2024. URL: <https://learn.microsoft.com/en-us/troubleshoot/windows-client/setup-upgrade-and-drivers/dynamic-link-library> (visited on 01/17/2025).
- [11] Brendan Dolan-Gavitt et al. “Repeatable Reverse Engineering with PANDA”. In: *Proceedings of the 5th Program Protection and Reverse Engineering Workshop*. PPREW-5. Los Angeles, CA, USA: Association for Computing Machinery, 2015. ISBN: 9781450336420. DOI: 10.1145/2843859.2843867. URL: <https://doi.org/10.1145/2843859.2843867>.
- [12] Eibe Frank, Mark A. Hall, and Ian H. Witten. *The WEKA Workbench. Online Appendix for "Data Mining: Practical Machine Learning Tools and Techniques"*. Fourth. Morgan Kaufmann, 2016.
- [13] Scott Freitas, Rahul Duggal, and Duen Horng Chau. *MalNet: A Large-Scale Image Database of Malicious Software*. 2022. arXiv: 2102.01072 [cs.CR]. URL: <https://arxiv.org/abs/2102.01072>.

- [14] Weijie Han et al. "MalDAE: Detecting and explaining malware based on correlation and fusion of static and dynamic characteristics". In: *Computers & Security* 83 (2019), pp. 208–233. ISSN: 0167-4048. DOI: <https://doi.org/10.1016/j.cose.2019.02.007>. URL: <https://www.sciencedirect.com/science/article/pii/S016740481831246X>.
- [15] Richard Harang and Ethan M. Rudd. *SOREL-20M: A Large Scale Benchmark Dataset for Malicious PE Detection*. 2020. arXiv: 2012.07634 [cs.CR].
- [16] Shamsul Huda et al. "A hybrid-multi filter-wrapper framework to identify run-time behaviour for fast malware detection". In: *Future Generation Computer Systems* 83 (June 1, 2018), pp. 193–207. ISSN: 0167-739X. DOI: 10.1016/j.future.2017.12.037. URL: <https://www.sciencedirect.com/science/article/pii/S0167739X17325049> (visited on 12/11/2024).
- [17] ElMouatez Billah Karbab et al. "MalDozer: Automatic framework for android malware detection using deep learning". In: *Digital Investigation* 24 (Mar. 1, 2018), S48–S59. ISSN: 1742-2876. DOI: 10.1016/j.diin.2018.01.007. URL: <https://www.sciencedirect.com/science/article/pii/S1742287618300392> (visited on 11/20/2024).
- [18] Karl-Bridge-Microsoft. *PE Format - Win32 apps*. Feb. 29, 2024. URL: <https://learn.microsoft.com/en-us/windows/win32/debug/pe-format> (visited on 01/17/2025).
- [19] Simon Kramer and Julian C. Bradfield. "A general definition of malware". In: *Journal in Computer Virology* 6.2 (May 1, 2010), pp. 105–114. ISSN: 1772-9904. DOI: 10.1007/s11416-009-0137-1. URL: <https://doi.org/10.1007/s11416-009-0137-1> (visited on 01/17/2025).
- [20] *Latest entries - The Portable Freeware Collection*. URL: <https://www.portablefreeware.com/> (visited on 01/17/2025).
- [21] Jure Leskovec, Anand Rajaraman, and Jeffrey David Ullman. *Mining of massive data sets*. Cambridge university press, 2020.
- [22] Ce Li et al. "DMalNet: Dynamic malware analysis based on API feature engineering and graph learning". In: *Computers & Security* 122 (Nov. 1, 2022), p. 102872. ISSN: 0167-4048. DOI: 10.1016/j.cose.2022.102872. URL: <https://www.sciencedirect.com/science/article/pii/S0167404822002668> (visited on 11/27/2024).
- [23] Hoda El Merabet and Abderrahmane Hajraoui. "A Survey of Malware Detection Techniques based on Machine Learning". In: *International Journal of Advanced Computer Science and Applications* 10.1 (2019). ISSN: 21565570, 2158107X. DOI: 10.14569 / IJACSA . 2019 . 0100148. URL: <http://thesai.org/Publications/ViewPaper?Volume=10&Issue=1&Code=ijacsa&SerialNo=48> (visited on 11/20/2024).
- [24] *microsoft/winget-cli*. original-date: 2019-07-16T22:16:49Z. Jan. 17, 2025. URL: <https://github.com/microsoft/winget-cli> (visited on 01/17/2025).
- [25] L. Nataraj et al. "Malware images: visualization and automatic classification". In: *Proceedings of the 8th International Symposium on Visualization for Cyber Security*. VizSec '11. Pittsburgh, Pennsylvania, USA: Association for Computing Machinery, 2011. ISBN: 9781450306799. DOI: 10.1145/2016904.2016908. URL: <https://doi.org/10.1145/2016904.2016908>.
- [26] National Audit Office (NAO). *Investigation: WannaCry cyber attack and the NHS*. London, England, Oct. 2017. URL: <https://web.archive.org/web/20241212140331/https://www.nao.org.uk/reports/investigation-wannacry-cyber-attack-and-the-nhs/>.
- [27] *Official Site | Malwarebytes: Antivirus, Anti-Malware & Privacy*. Malwarebytes. URL: <https://www.malwarebytes.com/> (visited on 01/16/2025).
- [28] Abdurrahman Pektaş and Tankut Acarman. "Classification of malware families based on runtime behaviors". In: *Journal of Information Security and Applications* 37 (2017), pp. 91–100. ISSN: 2214-2126. DOI: <https://doi.org/10.1016/j.jisa.2017.10.005>. URL: <https://www.sciencedirect.com/science/article/pii/S2214212617301643>.
- [29] *PortableApps.com - Portable software for USB, portable, and cloud drives*. PortableApps.com - Portable software for USB, portable, and cloud drives. URL: <https://portableapps.com/> (visited on 01/17/2025).
- [30] Edward Raff et al. "An investigation of byte n-gram features for malware classification". In: *Journal of Computer Virology and Hacking Techniques* 14.1 (Feb. 1, 2018), pp. 1–20. ISSN: 2263-8733. DOI: 10.1007/s11416-016-0283-1. URL: <https://doi.org/10.1007/s11416-016-0283-1>.
- [31] Guido Schwenk and Konrad Rieck. "Adaptive Detection of Covert Communication in HTTP Requests". In: *2011 Seventh European Conference on Computer Network Defense*. 2011, pp. 25–32. DOI: 10.1109/EC2ND.2011.12.
- [32] M. Shafiq et al. "PE-Miner: Mining Structural Information to Detect Malicious Executables in Realtime". In: Sept. 2009, pp. 121–141. ISBN: 978-3-642-04341-3. DOI: 10.1007/978-3-642-04342-0_7.

- [33] Jiawei Su et al. "Lightweight Classification of IoT Malware Based on Image Recognition". In: *2018 IEEE 42nd Annual Computer Software and Applications Conference (COMPSAC)*. Vol. 02. 2018, pp. 664–669. DOI: 10.1109/COMPSAC.2018.10315.
- [34] Umm-e-Hani Tayyab et al. "A Survey of the Recent Trends in Deep Learning Based Malware Detection". In: *Journal of Cybersecurity and Privacy* 2.4 (Dec. 2022). Number: 4 Publisher: Multidisciplinary Digital Publishing Institute, pp. 800–829. ISSN: 2624-800X. DOI: 10.3390/jcp2040041. URL: <https://www.mdpi.com/2624-800X/2/4/41> (visited on 11/20/2024).
- [35] Ronghua Tian et al. "An automated classification system based on the strings of trojan and virus families". In: *2009 4th International Conference on Malicious and Unwanted Software (MALWARE)*. 2009, pp. 23–30. DOI: 10.1109/MALWARE.2009.5403021.
- [36] Ashish Vaswani et al. *Attention Is All You Need*. 2023. arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- [37] R. Vinayakumar et al. "Robust Intelligent Malware Detection Using Deep Learning". In: *IEEE Access* 7 (2019). Conference Name: IEEE Access, pp. 46717–46738. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2019.2906934. URL: <https://ieeexplore.ieee.org/document/8681127/?arnumber=8681127> (visited on 11/20/2024).
- [38] *VirusShare.com*. URL: <https://virusshare.com/about> (visited on 01/16/2025).
- [39] Omar Yaacoubi. "The rise of encrypted malware". In: *Network Security* 2019.5 (2019), pp. 6–9. DOI: 10.1016/S1353-4858(19)30059-5. eprint: [https://doi.org/10.1016/S1353-4858\(19\)30059-5](https://doi.org/10.1016/S1353-4858(19)30059-5). URL: [https://doi.org/10.1016/S1353-4858\(19\)30059-5](https://doi.org/10.1016/S1353-4858(19)30059-5).
- [40] Ilsun You and Kangbin Yim. "Malware Obfuscation Techniques: A Brief Survey". In: *2010 International Conference on Broadband, Wireless Computing, Communication and Applications*. 2010, pp. 297–300. DOI: 10.1109/BWCCA.2010.85.